

Timezone Handling: transactionDate & warrantyEndDate

2026-07-17

Both fields are now genuinely correct UTC instants (same reliability as createdAt/updatedAt) - a backend bug that mislabeled Riyadh local time as UTC was just fixed. This page is only about how the iOS side should consume them correctly.

What to do

Parse both fields as standard ISO 8601 UTC and let Foundation render them in the device's own local timezone - no manual offset math needed on the Swift side, regardless of what timezone the device is in.

```
let formatter = ISO8601DateFormatter()
formatter.formatOptions = [.withInternetDateTime, .withFractionalSeconds]
let date = formatter.date(from: transactionDateString)
```

Concrete gotcha: these timestamps include milliseconds, e.g. "2026-07-12T08:15:00.000Z". Foundation's default ISO8601DateFormatter() (with no options set) does **not** parse the .000 fractional-seconds part and will silently return nil. Make sure .withFractionalSeconds is included in formatOptions, as shown above. If decoding through Codable instead, use JSONDecoder.DateDecodingStrategy.iso8601 (Swift 5.9+ handles fractional seconds there by default) rather than rolling a custom formatter per model.

What not to do

- Don't strip or ignore the trailing z and treat the string as if it were already local time.
- Don't hardcode a Riyadh (+03:00) offset on the client - the value is true UTC now, so correctly parsing it renders the right local time automatically, on any device, in any timezone.
- Both fields are optional - model them as Date?, they'll be null whenever the receipt didn't clearly print that information.

One thing to expect, not a new bug

Only receipts parsed *after* this fix shipped carry correct values. Anything parsed before that - including older test receipts - still carries the old, incorrect value (off by exactly 3 hours). If you spot a receipt whose displayed time looks exactly 3 hours off from when you'd expect, that's old data, not a client-side parsing mistake.